Student Management System

Complete Interview Preparation Guide — From R&D to Production

Spring Boot 3.3.0 | Java 17 | PostgreSQL | JPA / Hibernate | Maven

Table of Contents

1. Project Overview

This is a **RESTful Student Management System** built with Spring Boot. It provides CRUD (Create, Read, Update, Delete) operations for managing student records, along with search functionality.

Business Requirements:

- Store student information: roll number, name, email, phone, department, semester, CGPA
- Add new students
- Retrieve student details by ID
- List all students
- Update existing student records
- Delete student records
- Search students by name (partial match)
- Search students by department
- Search student by exact roll number

2. Tech Stack & Dependencies

Technology	Purpose
Java 17	Programming language
Spring Boot 3.3.0	Framework (auto-configuration, embedded server)
Spring Web	REST API (Tomcat embedded, @RestController, @RequestMapping)
Spring Data JPA	Database access via Hibernate ORM
PostgreSQL	Relational database
Lombok	Boilerplate code reduction (@Data, @NoArgsConstructor, etc.)
Bean Validation	Jakarta @NotBlank, @Email, @Min/@Max annotations
SpringDoc OpenAPI	Swagger UI for API documentation

JUnit 5 + Mockito	Unit & integration testing
Maven	Build tool & dependency management

Key pom.xml dependencies explained:

- `spring-boot-starter-web` — Brings embedded Tomcat, REST annotations, Jackson for JSON
- `spring-boot-starter-data-jpa` — Hibernate ORM + Spring Data JPA (auto-implements repositories)
- `postgresql` — PostgreSQL JDBC driver (runtime scope only)
- `spring-boot-starter-validation` — Jakarta Bean Validation API + Hibernate Validator
- `lombok` — Generates getters/setters/constructors at compile time
- `springdoc-openapi-starter-webmvc-ui` — Auto-generates Swagger docs from annotations

3. Project Structure (Layered Architecture)

```
Student_Management/
├── backend/
│   ├── pom.xml
│   └── src/main/java/com/studentmanagement/
│       ├── StudentManagementApplication.java   <-- Entry point (@SpringBootApplication)
│       ├── config/                              <-- Configuration classes (CORS, etc.)
│       ├── entity/Student.java                  <-- JPA entity (database table mapping)
│       ├── repository/StudentRepository.java    <-- Data access layer (Spring Data JPA)
│       ├── dto/StudentDTO.java                  <-- Data Transfer Object (API contract)
│       ├── mapper/StudentMapper.java            <-- Entity ⇔ DTO conversion
│       ├── exception/                           <-- Custom exceptions + global handler
│       ├── service/
│       │   ├── StudentService.java              <-- Service interface (contract)
│       │   └── impl/StudentServiceImpl.java     <-- Business logic implementation
│       └── controller/StudentController.java    <-- REST endpoints (@RestController)
│   └── src/test/java/com/studentmanagement/
│       ├── mapper/StudentMapperTest.java
│       ├── service/StudentServiceImplTest.java
│       └── controller/StudentControllerTest.java
```

Why Layered Architecture? Separation of concerns — each layer has a single responsibility. Changes in one layer don't affect others. Makes the code testable, maintainable, and scalable.

---

4. Layer 1: Entity — `Student.java`

Purpose: Maps the Java class to a database table using JPA annotations.

```
@Entity
@Table(name = "students")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Student {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "roll_number", nullable = false, unique = true, length = 20)
    private String rollNumber;

    @Column(name = "first_name", nullable = false, length = 50)
    private String firstName;

    @Column(name = "last_name", nullable = false, length = 50)
    private String lastName;

    @Column(nullable = false, unique = true, length = 100)
    private String email;

    @Column(length = 15)
    private String phone;

    @Column(nullable = false, length = 50)
    private String department;

    @Column(nullable = false)
    private Integer semester;

    @Column(nullable = false)
    private Double cgpa;
}
```

Annotation	Explanation
`@Entity`	Marks this class as a JPA entity (maps to a database table)

`@Table(name = "students")`	Specifies the table name in the database
`@Id`	Marks the primary key field
`@GeneratedValue(strategy = GenerationType.IDENTITY)`	Auto-increment ID — database generates the value
`@Column(name = "...", nullable = false, unique = true, length = 50)`	Configures column: name, NOT NULL, UNIQUE constraint, max length
`@Data` (Lombok)	Generates getters, setters, toString, equals, hashCode
`@NoArgsConstructor / @AllArgsConstructor`	Generates constructors (JPA requires no-arg constructor)

> **Interview Tip:** JPA uses the `@Entity` annotation to scan entities at startup. The `@Table` name defaults to the class name if not specified. `GenerationType.IDENTITY` relies on the database's auto-increment column (e.g., PostgreSQL SERIAL or IDENTITY). Hibernate DDL mode `update` in application.yml auto-creates/alters tables based on entity definitions — never use this in production!

5. Layer 2: Repository — `StudentRepository.java`

Purpose: Provides database CRUD operations without writing SQL. Spring Data JPA auto-implements this interface at runtime.

```
@Repository
public interface StudentRepository extends JpaRepository<Student, Long> {

    Optional<Student> findByRollNumber(String rollNumber);

    List<Student> findByFirstNameContainingIgnoreCaseOrLastNameContainingIgnoreCase(
            String firstName, String lastName);

    List<Student> findByDepartmentIgnoreCase(String department);
}
```

Concept	Explanation
`JpaRepository<Student, Long>`	Built-in CRUD: save(), findById(), findAll(), deleteById(), etc. First type = entity, second = ID type
`@Repository`	Marks as a Spring bean (exception translation for JDBC exceptions)
`findByRollNumber(String)`	**Query Derivation:** Spring Data JPA parses method name and generates JPQL automatically
`ContainingIgnoreCase`	Generates `WHERE LOWER(column) LIKE LOWER(CONCAT('%', ?, '%'))`
`Or`	Generates `WHERE condition1 OR condition2`
`Optional<T>`	Container that may or may not hold a value — avoids null pointer exceptions

Interview Tip: Query derivation keywords: `findBy`, `And`, `Or`, `Between`, `LessThan`, `GreaterThan`, `Like`, `Containing`, `IgnoreCase`, `OrderBy`. For complex queries, use `@Query` with JPQL or native SQL. `Optional` forces the caller to handle the absent case (better than returning null).

6. Layer 3: DTO — `StudentDTO.java`

Purpose: Data Transfer Object — defines the API contract (what the client sends/receives). Decouples the internal entity from the external API.

```
@Data
@NoArgsConstructor
@AllArgsConstructor
public class StudentDTO {

    private Long id;

    @NotBlank(message = "Roll number is required")
    private String rollNumber;

    @NotBlank(message = "First name is required")
    private String firstName;

    @NotBlank(message = "Last name is required")
    private String lastName;

    @NotBlank(message = "Email is required")
    @Email(message = "Email must be valid")
    private String email;

    @NotBlank(message = "Phone number is required")
    private String phone;

    @NotBlank(message = "Department is required")
    private String department;

    @NotNull(message = "Semester is required")
    @Min(value = 1, message = "Semester must be at least 1")
    @Max(value = 8, message = "Semester must be at most 8")
    private Integer semester;

    @NotNull(message = "CGPA is required")
    @DecimalMin(value = "0.0", message = "CGPA must be at least 0.0")
    @DecimalMax(value = "10.0", message = "CGPA must be at most 10.0")
    private Double cgpa;
}
```

Annotation	Explanation
`@NotBlank`	String must not be null and must contain at least one non-whitespace character
`@Email`	Validates email format (e.g., user@domain.com)
`@NotNull`	Value must not be null (for non-String types like Integer, Double)
`@Min / @Max`	Integer must be within the specified range (inclusive)
`@DecimalMin / @DecimalMax`	Decimal value must be within range (for Double/BigDecimal)

> **Interview Tip:** DTOs prevent entity exposure to clients. If the entity structure changes (e.g., adding an internal field), the API contract stays stable. Validation annotations trigger `MethodArgumentNotValidException` when `@Valid` is used in the controller.

7. Layer 4: Mapper — `StudentMapper.java`

Purpose: Converts between Entity and DTO. Keeps conversion logic centralized instead of spreading it across service/controller layers.

```
@Component
public class StudentMapper {

    public StudentDTO toDTO(Student student) {
        if (student == null) return null;
        StudentDTO dto = new StudentDTO();
        dto.setId(student.getId());
        dto.setRollNumber(student.getRollNumber());
        dto.setFirstName(student.getFirstName());
        dto.setLastName(student.getLastName());
        dto.setEmail(student.getEmail());
        dto.setPhone(student.getPhone());
        dto.setDepartment(student.getDepartment());
        dto.setSemester(student.getSemester());
        dto.setCgpa(student.getCgpa());
        return dto;
    }

    public Student toEntity(StudentDTO dto) {
        if (dto == null) return null;
        Student student = new Student();
        student.setId(dto.getId());
        student.setRollNumber(dto.getRollNumber());
        student.setFirstName(dto.getFirstName());
        student.setLastName(dto.getLastName());
        student.setEmail(dto.getEmail());
        student.setPhone(dto.getPhone());
        student.setDepartment(dto.getDepartment());
        student.setSemester(dto.getSemester());
        student.setCgpa(dto.getCgpa());
        return student;
    }
}
```

Annotation	Explanation
`@Component`	Marks as a Spring-managed bean (auto-detected during component scan)

Interview Tip: In larger projects, use **MapStruct** (annotation-based mapper) to auto-generate mapping code at compile time — eliminates boilerplate and is faster than manual mapping.

8. Layer 5: Exception Handling

Custom Exception — `StudentNotFoundException.java`

```
public class StudentNotFoundException extends RuntimeException {
    public StudentNotFoundException(Long id) {
        super("Student not found with id: " + id);
    }
    public StudentNotFoundException(String message) {
        super(message);
    }
}
```

Error Response DTO — `ErrorResponse.java`

```
@Data
@AllArgsConstructor
public class ErrorResponse {
    private int status;
    private String message;
    private LocalDateTime timestamp;
}
```

Global Exception Handler — `GlobalExceptionHandler.java`

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(StudentNotFoundException.class)
    public ResponseEntity<ErrorResponse> handleStudentNotFound(StudentNotFoundException e
        ErrorResponse error = new ErrorResponse(
                HttpStatus.NOT_FOUND.value(), ex.getMessage(), LocalDateTime.now());
        return new ResponseEntity<>(error, HttpStatus.NOT_FOUND);
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    public ResponseEntity<ErrorResponse> handleValidation(MethodArgumentNotValidException
        String message = ex.getBindingResult().getFieldErrors().stream()
                .map(e -> e.getField() + ": " + e.getDefaultMessage())
                .reduce((a, b) -> a + "; " + b)
                .orElse("Validation failed");
        ErrorResponse error = new ErrorResponse(
                HttpStatus.BAD_REQUEST.value(), message, LocalDateTime.now());
        return new ResponseEntity<>(error, HttpStatus.BAD_REQUEST);
    }

    @ExceptionHandler(Exception.class)
    public ResponseEntity<ErrorResponse> handleGeneric(Exception ex) {
        ErrorResponse error = new ErrorResponse(
                HttpStatus.INTERNAL_SERVER_ERROR.value(),
                "An unexpected error occurred", LocalDateTime.now());
        return new ResponseEntity<>(error, HttpStatus.INTERNAL_SERVER_ERROR);
    }
}
```

Annotation	Explanation
`@RestControllerAdvice`	Global exception handler for all @RestController classes — AOP-based
`@ExceptionHandler(StudentNotFoundException.class)`	Handles specific exception type; returns custom response with

	correct HTTP status
`MethodArgumentNotValidException`	Thrown automatically when @Valid fails on @RequestBody
`ex.getBindingResult().getFieldErrors()`	Contains all validation errors with field names and default messages

> **Interview Tip:** Centralized exception handling avoids try-catch blocks in controllers. `@RestControllerAdvice` is a combination of `@ControllerAdvice` + `@ResponseBody`. Always return a consistent error structure (`ErrorResponse`) so clients can parse errors uniformly.

9. Layer 6: Service — Business Logic

Purpose: Contains all business rules. The service layer coordinates between the controller (incoming requests) and repository (database).

Interface — `StudentService.java`

```
public interface StudentService {
    StudentDTO addStudent(StudentDTO studentDTO);
    StudentDTO getStudentById(Long id);
    List<StudentDTO> getAllStudents();
    StudentDTO updateStudent(Long id, StudentDTO studentDTO);
    void deleteStudent(Long id);
    List<StudentDTO> searchStudentByName(String name);
    List<StudentDTO> searchStudentByDepartment(String department);
    StudentDTO searchStudentByRollNumber(String rollNumber);
}
```

Implementation — `StudentServiceImpl.java`

```
@Service
@RequiredArgsConstructor
public class StudentServiceImpl implements StudentService {

    private final StudentRepository studentRepository;
    private final StudentMapper studentMapper;

    @Override
    public StudentDTO addStudent(StudentDTO studentDTO) {
        Student student = studentMapper.toEntity(studentDTO);
        Student savedStudent = studentRepository.save(student);
        return studentMapper.toDTO(savedStudent);
    }

    @Override
    public StudentDTO getStudentById(Long id) {
        Student student = studentRepository.findById(id)
                .orElseThrow(() -> new StudentNotFoundException(id));
        return studentMapper.toDTO(student);
    }

    @Override
    public List<StudentDTO> getAllStudents() {
        return studentRepository.findAll().stream()
                .map(studentMapper::toDTO)
                .collect(Collectors.toList());
    }

    @Override
    public StudentDTO updateStudent(Long id, StudentDTO studentDTO) {
        Student existing = studentRepository.findById(id)
                .orElseThrow(() -> new StudentNotFoundException(id));
        existing.setRollNumber(studentDTO.getRollNumber());
        existing.setFirstName(studentDTO.getFirstName());
        existing.setLastName(studentDTO.getLastName());
        existing.setEmail(studentDTO.getEmail());
        existing.setPhone(studentDTO.getPhone());
        existing.setDepartment(studentDTO.getDepartment());
        existing.setSemester(studentDTO.getSemester());
        existing.setCgpa(studentDTO.getCgpa());
        return studentMapper.toDTO(studentRepository.save(existing));
```

```
    }

    @Override
    public void deleteStudent(Long id) {
        Student student = studentRepository.findById(id)
                .orElseThrow(() -> new StudentNotFoundException(id));
        studentRepository.delete(student);
    }

    @Override
    public StudentDTO searchStudentByRollNumber(String rollNumber) {
        Student student = studentRepository.findByRollNumber(rollNumber)
                .orElseThrow(() -> new StudentNotFoundException(
                        "Student not found with roll number: " + rollNumber));
        return studentMapper.toDTO(student);
    }
}
```

Annotation	Explanation
`@Service`	Marks as service layer bean (specialization of @Component)
`@RequiredArgsConstructor`	Lombok: generates constructor for all `final` fields (constructor injection)
`@Override`	Ensures method is implementing an interface method (compiler check)
`.orElseThrow()`	If Optional is empty, throws the supplier exception — clean error handling

Interview Tip: Always program to interfaces (`StudentService` interface, not the impl). This enables loose coupling — you can swap implementations (e.g., a mock for testing) without changing consumers. Constructor injection with `final` fields ensures immutability and makes dependencies explicit.

10. Layer 7: Controller — REST API

Purpose: Handles HTTP requests, delegates to service layer, and returns HTTP responses.

```
@RestController
@RequestMapping("/api/students")
@RequiredArgsConstructor
@CrossOrigin(origins = "*")
public class StudentController {

    private final StudentService studentService;

    @PostMapping
    public ResponseEntity<StudentDTO> addStudent(@Valid @RequestBody StudentDTO studentDT
        StudentDTO created = studentService.addStudent(studentDTO);
        return new ResponseEntity<>(created, HttpStatus.CREATED);
    }

    @GetMapping("/{id}")
    public ResponseEntity<StudentDTO> getStudentById(@PathVariable Long id) {
        return ResponseEntity.ok(studentService.getStudentById(id));
    }

    @GetMapping
    public ResponseEntity<List<StudentDTO>> getAllStudents() {
        return ResponseEntity.ok(studentService.getAllStudents());
    }

    @PutMapping("/{id}")
    public ResponseEntity<StudentDTO> updateStudent(
            @PathVariable Long id, @Valid @RequestBody StudentDTO studentDTO) {
        return ResponseEntity.ok(studentService.updateStudent(id, studentDTO));
    }

    @DeleteMapping("/{id}")
    public ResponseEntity<Void> deleteStudent(@PathVariable Long id) {
        studentService.deleteStudent(id);
        return ResponseEntity.noContent().build();
    }

    @GetMapping("/search/name")
    public ResponseEntity<List<StudentDTO>> searchStudentByName(@RequestParam String name
        return ResponseEntity.ok(studentService.searchStudentByName(name));
    }

    @GetMapping("/search/department")
    public ResponseEntity<List<StudentDTO>> searchStudentByDepartment(@RequestParam Strin
```

```
        return ResponseEntity.ok(studentService.searchStudentByDepartment(department));
    }

    @GetMapping("/search/roll-number")
    public ResponseEntity<StudentDTO> searchStudentByRollNumber(@RequestParam String roll
        return ResponseEntity.ok(studentService.searchStudentByRollNumber(rollNumber));
    }
}
```

Annotation	Explanation
`@RestController`	= @Controller + @ResponseBody (every method returns JSON, not view name)
`@RequestMapping("/api/students")`	Base URL path for all endpoints in this controller
`@PostMapping`	HTTP POST — for creating resources
`@GetMapping`	HTTP GET — for reading/retrieving resources
`@PutMapping`	HTTP PUT — for updating existing resources (full replacement)
`@DeleteMapping`	HTTP DELETE — for removing resources
`@PathVariable`	Extracts value from URL path: `/api/students/{id}`
`@RequestParam`	Extracts query parameter: `?name=John`
`@RequestBody`	Binds JSON body to Java object (Jackson serialization)
`@Valid`	Triggers validation on @RequestBody before method executes
`@CrossOrigin`	Allows CORS (Cross-Origin Resource Sharing) — needed for frontend integration
`ResponseEntity<T>`	Full HTTP response: body, status code, headers

Interview Tip: `ResponseEntity` gives full control over HTTP response. `ResponseEntity.ok()` = 200, `ResponseEntity.noContent()` = 204, `new ResponseEntity<>(body, HttpStatus.CREATED)` = 201. RESTful conventions: POST = create (201), GET = read (200), PUT = update (200), DELETE = delete (204).

11. Layer 8: Configuration — `application.yml`

```yaml
server:
  port: 8080

spring:
  datasource:
    url: jdbc:postgresql://localhost:5432/student_management_db
    username: postgres
    password: postgres
    driver-class-name: org.postgresql.Driver

  jpa:
    hibernate:
      ddl-auto: update
    show-sql: true
    properties:
      hibernate:
        dialect: org.hibernate.dialect.PostgreSQLDialect

springdoc:
  api-docs:
    path: /api-docs
  swagger-ui:
    path: /swagger-ui.html
```

Property	Explanation
`server.port`	Port on which the embedded Tomcat runs
`spring.datasource.url`	JDBC connection URL: `jdbc:postgresql://host:port/database_name`

`spring.jpa.hibernate.ddl-auto: update`	Hibernate auto-creates/updates tables based on entities. Options: `none` (prod), `validate`, `update`, `create`, `create-drop`
`spring.jpa.show-sql: true`	Logs SQL queries (for debugging; disable in production)
`hibernate.dialect`	Optimizes SQL generation for PostgreSQL
`springdoc`	Swagger/OpenAPI configuration paths

---

12. All REST API Endpoints

Method	Endpoint	Description	Request Body	Response
POST	`/api/students`	Add a new student	StudentDTO JSON	201 Created + StudentDTO
GET	`/api/students/{id}`	Get student by ID	-	200 OK + StudentDTO
GET	`/api/students`	Get all students	-	200 OK + StudentDTO[]
PUT	`/api/students/{id}`	Update existing student	StudentDTO JSON	200 OK + StudentDTO
DELETE	`/api/students/{id}`	Delete student by ID	-	204 No Content
GET	`/api/students/search/name?name=John`	Search by name	-	200 OK + StudentDTO[]

		(partial match)		
GET	`/api/students/search/department?department=CS`	Search by department	-	200 OK + StudentDTO[]
GET	`/api/students/search/roll-number?rollNumber=S123`	Search by exact roll number	-	200 OK + StudentDTO

13. Validation & Error Responses

Sample Error Responses:

404 — Student Not Found:

```
{
  "status": 404,
  "message": "Student not found with id: 115",
  "timestamp": "2026-06-14T15:30:00.000"
}
```

400 — Validation Failed:

```
{
  "status": 400,
  "message": "firstName: First name is required; email: Email must be valid; semester: Se
  "timestamp": "2026-06-14T15:30:00.000"
}
```

500 — Internal Server Error:

```
{
  "status": 500,
  "message": "An unexpected error occurred",
  "timestamp": "2026-06-14T15:30:00.000"
}
```

14. Testing Strategy

Test Coverage (28 tests):

Test Class	Tests	Type	What it tests
`StudentMapperTest`	4	Unit	Mapping logic: Entity ↔ DTO conversion, null handling
`StudentServiceImplTest`	12	Unit (Mockito)	Business logic: CRUD operations, exceptions, search with mocked repository
`StudentControllerTest`	12	Integration (@WebMvcTest)	HTTP endpoints: status codes, request/response JSON, validation errors

Testing Annotations:

- `@ExtendWith(MockitoExtension.class)` — Enables Mockito in JUnit 5
- `@Mock` — Creates a mock (fake) object
- `@WebMvcTest(StudentController.class)` — Loads only controller layer (lightweight)
- `@MockBean` — Replaces actual Spring bean with mock in application context
- `MockMvc` — Simulates HTTP requests without starting the server

15. Database Schema

```
Table: students
+----------------+--------------+--------+
| Column         | Type         | Constraint      |
+----------------+--------------+--------+
| id             | BIGSERIAL    | PRIMARY KEY     |
| roll_number    | VARCHAR(20)  | NOT NULL, UNIQUE|
| first_name     | VARCHAR(50)  | NOT NULL        |
| last_name      | VARCHAR(50)  | NOT NULL        |
| email          | VARCHAR(100) | NOT NULL, UNIQUE|
| phone          | VARCHAR(15)  |                 |
| department     | VARCHAR(50)  | NOT NULL        |
| semester       | INTEGER      | NOT NULL        |
| cgpa           | DOUBLE       | NOT NULL        |
+----------------+--------------+--------+
```

16. How to Run

1. **Prerequisites:**
   - Java 17+
   - Maven 3.8+
   - PostgreSQL (running on localhost:5432)
2. **Create Database:**

```
CREATE DATABASE student_management_db;
```

3. **Navigate to backend:**

```
cd backend
```

4. **Run the application:**

```
mvn spring-boot:run
```

5. **Access Swagger UI:**

```
http://localhost:8080/swagger-ui.html
```

6. **Run tests:**

```
mvn test
```

17. Postman Collection

Base URL: `http://localhost:8080`

1. Add Student — POST

POST `http://localhost:8080/api/students`

```
Headers: Content-Type: application/json

Body (JSON):
{
  "rollNumber": "S123",
  "firstName": "John",
  "lastName": "Doe",
  "email": "john@example.com",
  "phone": "1234567890",
  "department": "Computer Science",
  "semester": 3,
  "cgpa": 8.5
}
```

2. Get All Students — GET

GET `http://localhost:8080/api/students`

3. Get Student by ID — GET

GET `http://localhost:8080/api/students/1`

4. Update Student — PUT

PUT `http://localhost:8080/api/students/1`

```
Body (same structure as POST)
```

5. Delete Student — DELETE

DELETE `http://localhost:8080/api/students/1`

6. Search by Name — GET

GET `http://localhost:8080/api/students/search/name?name=John`

7. Search by Department — GET

GET `http://localhost:8080/api/students/search/department?department=CS`

8. Search by Roll Number — GET

GET `http://localhost:8080/api/students/search/roll-number?rollNumber=S123`

18. Interview Questions & Answers

Q1: What is the difference between @Entity, @Table, and @Column?

@Entity marks the class as a JPA entity that maps to a database table. **@Table** specifies the table name (optional; defaults to class name). **@Column** configures individual column properties like name, nullable, unique, length.

Q2: Why use DTOs instead of exposing entities directly?

DTOs decouple the API contract from the internal persistence model. If the entity changes (e.g., adding a password hash), the API stays the same. DTOs also prevent lazy loading issues and over-fetching of data.

Q3: What is the purpose of @RestControllerAdvice?

It provides centralized exception handling across all @RestController classes. Instead of try-catch in every controller method, one class handles all exceptions and returns consistent error responses.

Q4: Explain @PathVariable vs @RequestParam

@PathVariable extracts values from the URL path template (e.g., `/students/{id}` -> id=5). **@RequestParam** extracts query parameters from the URL after `?` (e.g., `?name=John` -> name=John). Use path variables for required resource identifiers, query params for optional filters.

Q5: How does Spring Data JPA's query derivation work?

Spring Boot parses the method name at startup and generates JPQL queries automatically. For example, `findByFirstNameContainingIgnoreCaseOrLastNameContainingIgnoreCase(String, String)` generates: `WHERE LOWER(firstName) LIKE LOWER(CONCAT('%', ?, '%')) OR LOWER(lastName) LIKE LOWER(CONCAT('%', ?, '%'))`.

Q6: What is the difference between @Mock and @MockBean?

@Mock (Mockito) creates a simple mock — used in plain unit tests. **@MockBean** (Spring Boot) creates a mock and replaces the actual bean in the Spring application context — used in integration tests like @WebMvcTest.

Q7: What HTTP status codes should a REST API return?

POST (create) → 201 Created. GET (read) → 200 OK. PUT (update) → 200 OK. DELETE (delete) → 204 No Content. Validation errors → 400 Bad Request. Not found → 404. Server errors → 500.

Q8: Explain the layered architecture pattern

- **Controller** — Handles HTTP, delegates to service, returns response
- **Service** — Business logic, transactions, coordinates other layers
- **Repository** — Data access, queries
- **Entity** — Database mapping
- **DTO** — API contract

Each layer only talks to the layer directly below it. This creates a separation of concerns and makes the code testable.

Q9: How would you handle updating only specific fields (PATCH vs PUT)?

PUT replaces the entire resource — client sends all fields. PATCH is for partial updates — client sends only changed fields. For PATCH, you'd typically read the existing entity, apply only non-null fields from the request, and save.

Q10: What is Optional and why use it?

Optional is a container that may or may not hold a value. It forces the caller to handle the empty case explicitly (using `orElse()`, `orElseThrow()`, etc.) instead of risking NullPointerException from returning null.

Student Management System — Complete Interview Guide